NetApp Astra Trident 概述

NetApp Solutions

NetApp
May 17, 2022

This PDF was generated from https://docs.netapp.com/zh-cn/netapp-solutions/containers/anthos-with-netapp/a-w-n_trident_ontap_nfs.html on May 17, 2022. Always check docs.netapp.com for the latest.

目录

Astra Trident 概述 1
下载 Astra Trident 1
使用 Helm 安装 Trident 操作员 2
手动安装 Trident 操作员 4
创建存储系统后端 8
NetApp ONTAP NFS配置：采用NetApp的Anthos 8
NetApp ONTAP iSCSI配置：采用NetApp的Anthos 10
NetApp Element iSCSI配置：采用NetApp的Anthos 13

Astra Trident 概述

Astra Trident是一款开源且受完全支持的存储编排程序、适用于容器和Kubernetes分发版、包括Anthos。Trident 可与包括 NetApp ONTAP 和 Element 存储系统在内的整个 NetApp 存储产品组合配合使用，并且还支持 NFS 和 iSCSI 连接。Trident 允许最终用户从其 NetApp 存储系统配置和管理存储，而无需存储管理员干预，从而加快了 DevOps 工作流的速度。

管理员可以根据项目需求和存储系统型号配置多个存储后端，以实现高级存储功能，包括数据压缩，特定磁盘类型或 QoS 级别，以保证一定水平的性能。定义后，开发人员可以在其项目中使用这些后端创建永久性卷声明（PVC），并按需将永久性存储附加到容器。

[错误：缺少图形映像]

Astra Trident具有快速的开发周期、与Kubernetes一样、每年发布四次。

最新版Astra Trident 22.01于2022年1月发布。已测试的 Trident 版本的支持列表，可在该支持列表中找到 Kubernetes 分发版本 "此处"。

从 20.04 版开始， Trident 设置由 Trident 操作员执行。操作员可以简化大规模部署，并为在 Trident 安装过程中部署的 Pod 提供额外的支持，包括自我修复。

在 21.01 版中，我们提供了一个 Helm 图表，用于简化 Trident 操作员的安装。

下载 Astra Trident

要在已部署的用户集群上安装 Trident 并配置永久性卷，请完成以下步骤：

1. 将安装归档下载到管理工作站并提取内容。Trident 的当前版本为 22.01 ，可以下载 "此处"。

```
[ubuntu@gke-admin-ws-2021-07-15 ~]$ wget
https://github.com/NetApp/trident/releases/download/v22.01.0/trident-
installer-22.01.0.tar.gz
--2021-05-06 15:17:30--
https://github.com/NetApp/trident/releases/download/v22.01.0/trident-
installer-22.01.0.tar.gz
Resolving github.com (github.com)... 140.82.114.3
Connecting to github.com (github.com)|140.82.114.3|:443... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://github-
releases.githubusercontent.com/77179634/a4fa9f00-a9f2-11eb-9053-
98e8e573d4ae?X-Amz-Algorithm=AWS4-HMAC-SHA256&X-Amz-
Credential=AKIAIWNJYAX4CSVEH53A%2F20210506%2Fus-east-
1%2Fs3%2Faws4_request&X-Amz-Date=20210506T191643Z&X-Amz-Expires=300&X-
Amz-
Signature=8a49a2a1e08c147d1ddd8149ce45a5714f9853fee19bb1c507989b9543eb36
30&X-Amz-
SignedHeaders=host&actor_id=0&key_id=0&repo_id=77179634&response-
content-disposition=attachment%3B%20filename%3Dtrident-installer-
```

```
22.01.0.tar.gz&response-content-type=application%2Foctet-stream
[following]
--2021-05-06 15:17:30--  https://github-
releases.githubusercontent.com/77179634/a4fa9f00-a9f2-11eb-9053-
98e8e573d4ae?X-Amz-Algorithm=AWS4-HMAC-SHA256&X-Amz-
Credential=AKIAIWNJYAX4CSVEH53A%2F20210506%2Fus-east-
1%2Fs3%2Faws4_request&X-Amz-Date=20210506T191643Z&X-Amz-Expires=300&X-
Amz-
Signature=8a49a2a1e08c147d1ddd8149ce45a5714f9853fee19bb1c507989b9543eb36
30&X-Amz-
SignedHeaders=host&actor_id=0&key_id=0&repo_id=77179634&response-
content-disposition=attachment%3B%20filename%3Dtrident-installer-
22.01.0.tar.gz&response-content-type=application%2Foctet-stream
Resolving github-releases.githubusercontent.com (github-
releases.githubusercontent.com)... 185.199.108.154, 185.199.109.154,
185.199.110.154, ...
Connecting to github-releases.githubusercontent.com (github-
releases.githubusercontent.com)|185.199.108.154|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 38349341 (37M) [application/octet-stream]
Saving to: 'trident-installer-22.01.0.tar.gz'

100%[=================================================================
==============================================>] 38,349,341  88.5MB/s
in 0.4s

2021-05-06 15:17:30 (88.5 MB/s) - 'trident-installer-22.01.0.tar.gz'
saved [38349341/38349341]
```

2. 从下载的软件包中提取 Trident 安装。

```
[ubuntu@gke-admin-ws-2021-07-15 ~]$ tar -xzf trident-installer-
22.01.0.tar.gz
[ubuntu@gke-admin-ws-2021-07-15 ~]$ cd trident-installer/
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$
```

使用 Helm 安装 Trident 操作员

> 默认情况下、GKE-Admin工作站上不安装Helm。您可以使用Ubuntu中提供的apt工具轻松安装它。

1. 首先，将用户集群的 `kubeconfig` 文件的位置设置为环境变量，以便您不必引用该文件，因为 Trident 没有传递此文件的选项。

```
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ export
KUBECONFIG=~/user-cluster-1/user-cluster-1-kubeconfig
```

2. 在用户集群中创建 Trident 命名空间时，运行 Helm 命令从 Helm 目录中的 tarball 安装 Trident 操作员。

```
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ helm install trident
helm/trident-operator-22.01.0.tgz --create-namespace --namespace trident
NAME: trident
LAST DEPLOYED: Fri May  7 12:54:25 2021
NAMESPACE: trident
STATUS: deployed
REVISION: 1
TEST SUITE: None
NOTES:
Thank you for installing trident-operator, which will deploy and manage
NetApp's Trident CSI
storage provisioner for Kubernetes.

Your release is named 'trident' and is installed into the 'trident'
namespace.
Please note that there must be only one instance of Trident (and
trident-operator) in a Kubernetes cluster.

To configure Trident to manage storage resources, you will need a copy
of tridentctl, which is
available in pre-packaged Trident releases.  You may find all Trident
releases and source code
online at https://github.com/NetApp/trident.

To learn more about the release, try:

  $ helm status trident
  $ helm get all trident
```

3. 您可以通过检查命名空间中运行的 Pod 或使用 tridentctl 二进制文件检查已安装的版本来验证 Trident 是否已成功安装。

```
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ oc get pods -n
trident
NAME                                READY   STATUS    RESTARTS   AGE
trident-csi-5z45l                   1/2     Running   2          30s
trident-csi-696b685cf8-htdb2        6/6     Running   0          30s
trident-csi-b74p2                   2/2     Running   0          30s
trident-csi-lrw4n                   2/2     Running   0          30s
trident-operator-7c748d957-gr2gw    1/1     Running   0          36s

[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ ./tridentctl -n
trident version
+----------------+----------------+
| SERVER VERSION | CLIENT VERSION |
+----------------+----------------+
| 22.01.0          | 22.01.0          |
+----------------+----------------+
```

在某些情况下，客户环境可能需要自定义 Trident 部署。在这种情况下，还可以手动安装 Trident 操作员并更新所包含的清单以自定义部署。

手动安装 Trident 操作员

1. 首先，将用户集群的 `kubeconfig` 文件的位置设置为环境变量，以便您不必引用该文件，因为 Trident 没有传递此文件的选项。

```
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ KUBECONFIG=~/user-
cluster-1/user-cluster-1-kubeconfig
```

2. `trident` **安装程序** 目录包含用于定义所有所需资源的清单。使用适当的清单创建 `TridentOrchestrator` 自定义资源定义。

```
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ kubectl create -f
deploy/crds/trident.netapp.io_tridentorchestrators_crd_post1.16.yaml
customresourcedefinition.apiextensions.k8s.io/tridentorchestrators.tride
nt.netapp.io created
```

3. 如果不存在 Trident 命名空间，请使用提供的清单在集群中创建一个 Trident 命名空间。

```
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ kubectl apply -f
deploy/namespace.yaml
namespace/trident created
```

4. 为 Trident 操作员部署创建所需的资源，例如为操作员创建 `ServiceAccount` ，为 `SClusterRole` 和 `ClusterRoleBinding` ，为` erviceAccount ，专用 PodSecurityPolicy `或操作员本身创建。

```
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ kubectl create -f
deploy/bundle.yaml
serviceaccount/trident-operator created
clusterrole.rbac.authorization.k8s.io/trident-operator created
clusterrolebinding.rbac.authorization.k8s.io/trident-operator created
deployment.apps/trident-operator created
podsecuritypolicy.policy/tridentoperatorpods created
```

5. 您可以使用以下命令在操作员部署后检查其状态：

```
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ kubectl get
deployment -n trident
NAME               READY   UP-TO-DATE   AVAILABLE   AGE
trident-operator   1/1     1            1           23s
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ kubectl get pods -n
trident
NAME                                READY   STATUS    RESTARTS   AGE
trident-operator-66f48895cc-lzczk   1/1     Running   0          41s
```

6. 部署操作员后，我们现在可以使用它来安装 Trident 。这需要创建 `TridentOrchestrator` 。

```
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ kubectl create -f
deploy/crds/tridentorchestrator_cr.yaml
tridentorchestrator.trident.netapp.io/trident created
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ kubectl describe
torc trident
Name:         trident
Namespace:
Labels:       <none>
Annotations:  <none>
API Version:  trident.netapp.io/v1
Kind:         TridentOrchestrator
Metadata:
  Creation Timestamp:  2021-05-07T17:00:28Z
  Generation:          1
  Managed Fields:
    API Version:  trident.netapp.io/v1
    Fields Type:  FieldsV1
    fieldsV1:
      f:spec:
        .:
```

```
        f:debug:
        f:namespace:
    Manager:      kubectl-create
    Operation:    Update
    Time:         2021-05-07T17:00:28Z
    API Version:  trident.netapp.io/v1
    Fields Type:  FieldsV1
    fieldsV1:
      f:status:
        .:
        f:currentInstallationParams:
          .:
          f:IPv6:
          f:autosupportHostname:
          f:autosupportImage:
          f:autosupportProxy:
          f:autosupportSerialNumber:
          f:debug:
          f:enableNodePrep:
          f:imagePullSecrets:
          f:imageRegistry:
          f:k8sTimeout:
          f:kubeletDir:
          f:logFormat:
          f:silenceAutosupport:
          f:tridentImage:
        f:message:
        f:namespace:
        f:status:
        f:version:
    Manager:         trident-operator
    Operation:       Update
    Time:            2021-05-07T17:00:28Z
  Resource Version:  931421
  Self Link:
/apis/trident.netapp.io/v1/tridentorchestrators/trident
  UID:               8a26a7a6-dde8-4d55-9b66-a7126754d81f
Spec:
  Debug:      true
  Namespace:  trident
Status:
  Current Installation Params:
    IPv6:                       false
    Autosupport Hostname:
    Autosupport Image:          netapp/trident-autosupport:21.01
    Autosupport Proxy:
```

```
    Autosupport Serial Number:
    Debug:                      true
    Enable Node Prep:           false
    Image Pull Secrets:
    Image Registry:
    k8sTimeout:           30
    Kubelet Dir:          /var/lib/kubelet
    Log Format:           text
    Silence Autosupport:  false
    Trident Image:        netapp/trident:22.01.0
  Message:                Trident installed
  Namespace:              trident
  Status:                 Installed
  Version:                v22.01.0
Events:
  Type    Reason      Age   From                        Message
  ----    ------      ----  ----                        -------
  Normal  Installing  80s   trident-operator.netapp.io  Installing
Trident
  Normal  Installed   68s   trident-operator.netapp.io  Trident
installed
```

7. 您可以通过检查命名空间中运行的 Pod 或使用 tridentctl 二进制文件检查已安装的版本来验证 Trident 是否已成功安装。

```
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ kubectl get pods -n
trident
NAME                                READY   STATUS    RESTARTS   AGE
trident-csi-bb64c6cb4-lmd6h         6/6     Running   0          82s
trident-csi-gn59q                   2/2     Running   0          82s
trident-csi-m4szj                   2/2     Running   0          82s
trident-csi-sb9k9                   2/2     Running   0          82s
trident-operator-66f48895cc-lzczk   1/1     Running   0          2m39s

[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ ./tridentctl -n
trident version
+----------------+----------------+
| SERVER VERSION | CLIENT VERSION |
+----------------+----------------+
| 22.01.0          | 22.01.0          |
+----------------+----------------+
```

创建存储系统后端

完成 Astra Trident 操作员安装后，您必须为所使用的特定 NetApp 存储平台配置后端。请访问以下链接继续设置和配置 Astra Trident 。

- "NetApp ONTAP NFS"
- "NetApp ONTAP iSCSI"
- "NetApp Element iSCSI"

"接下来：高级配置选项：采用NetApp的Anthos。"

NetApp ONTAP NFS配置：采用NetApp的Anthos

要启用 Trident 与 NetApp ONTAP 存储系统的集成，您必须创建一个后端，以便与存储系统进行通信。

1. 下载的安装归档中提供了 `sample-input` folder 层次结构中的示例后端文件。对于提供 NFS 的 NetApp ONTAP 系统，将 `backend-ontap-nas.json` 文件复制到您的工作目录并编辑该文件。

```
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ cp sample-
input/backends-samples/ontap-nas/backend-ontap-nas.json ./
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ vi backend-ontap-
nas.json
```

2. 编辑 backendName ， managementLIF ， dataLIF ， SVM ，用户名， 和密码值。

```
{
    "version": 1,
    "storageDriverName": "ontap-nas",
    "backendName": "ontap-nas+10.61.181.221",
    "managementLIF": "172.21.224.201",
    "dataLIF": "10.61.181.221",
    "svm": "trident_svm",
    "username": "cluster-admin",
    "password": "password"
}
```

> 最佳做法是，将自定义 backendName 值定义为 storageDriverName 和为 NFS 提供服务的 dataLIF 的组合，以便于识别。

3. 安装此后端文件后，运行以下命令以创建第一个后端。

```
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ ./tridentctl -n
trident create backend -f backend-ontap-nas.json
+------------------------+----------------
+--------------------------------------+--------+---------+
|          NAME          | STORAGE DRIVER |                  UUID
| STATE  | VOLUMES |
+------------------------+----------------
+--------------------------------------+--------+---------+
| ontap-nas+10.61.181.221 | ontap-nas      | be7a619d-c81d-445c-b80c-
5c87a73c5b1e | online |       0 |
+------------------------+----------------
+--------------------------------------+--------+---------+
```

4. 创建后端后，您接下来必须创建一个存储类。与后端一样，可以在 sample-inputs 文件夹中为环境编辑一个示例存储类文件。将其复制到工作目录并进行必要的编辑，以反映所创建的后端。

```
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ cp sample-
input/storage-class-samples/storage-class-csi.yaml.templ ./storage-
class-basic.yaml
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ vi storage-class-
basic.yaml
```

5. 必须对此文件进行的唯一编辑是，为新创建的后端存储驱动程序的名称定义 `backendType` 值。另请注意 name-field 值，稍后必须引用该值。

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: basic-csi
provisioner: csi.trident.netapp.io
parameters:
  backendType: "ontap-nas"
```

> 此文件中定义了一个名为 `FSType` 的可选字段。可以在 NFS 后端删除此行。

6. 运行 `kubectl` 命令以创建存储类。

```
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ kubectl create -f
storage-class-basic.yaml
storageclass.storage.k8s.io/basic-csi created
```

7. 创建存储类后，您必须创建第一个永久性卷请求（PVC）。此外，还可以在 sample-inputs 中使用一个示

例 `pva-basic` 。 `yaml` file 来执行此操作。

```
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ cp sample-input/pvc-
samples/pvc-basic.yaml ./
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ vi pvc-basic.yaml
```

8. 必须对此文件进行的唯一编辑是，确保 `storageClassName` 字段与刚刚创建的字段匹配。可以根据要配置的工作负载的需要进一步自定义 PVC 定义。

```
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: basic
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
  storageClassName: basic-csi
```

9. 发出 `kubectl` 命令创建 PVC 。根据所创建的后备卷的大小，创建可能需要一些时间，因此您可以在该过程完成后进行观察。

```
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ kubectl create -f
pvc-basic.yaml
persistentvolumeclaim/basic created

[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ kubectl get pvc
NAME     STATUS   VOLUME                                     CAPACITY
ACCESS MODES   STORAGECLASS   AGE
basic    Bound    pvc-b4370d37-0fa4-4c17-bd86-94f96c94b42d   1Gi
RWO            basic-csi      7s
```

"接下来：高级配置选项：采用NetApp的Anthos。"

NetApp ONTAP iSCSI配置：采用NetApp的Anthos

要启用 Trident 与 NetApp ONTAP 存储系统的集成，您必须创建一个后端，以便与存储系统进行通信。

1. 下载的安装归档中提供了 `sample-input` folder 层次结构中的示例后端文件。对于提供 iSCSI 的 NetApp ONTAP 系统，将 `backend-ontap-san.json` 文件复制到您的工作目录并编辑该文件。

```
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ cp sample-
input/backends-samples/ontap-san/backend-ontap-san.json ./
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ vi backend-ontap-
san.json
```

2. 编辑此文件中的 managementLIF ， dataLIF ， SVM ，用户名和密码值。

```
{
  "version": 1,
  "storageDriverName": "ontap-san",
  "managementLIF": "172.21.224.201",
  "dataLIF": "10.61.181.240",
  "svm": "trident_svm",
  "username": "admin",
  "password": "password"
}
```

3. 安装此后端文件后，运行以下命令以创建第一个后端。

```
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ ./tridentctl -n
trident create backend -f backend-ontap-san.json
+------------------------+----------------
+--------------------------------------+--------+---------+
|          NAME          | STORAGE DRIVER |                  UUID
| STATE  | VOLUMES |
+------------------------+----------------
+--------------------------------------+--------+---------+
| ontapsan_10.61.181.241 | ontap-san      | 6788533c-7fea-4a35-b797-
fb9bb3322b91 | online |       0 |
+------------------------+----------------
+--------------------------------------+--------+---------+
```

4. 创建后端后，您接下来必须创建一个存储类。与后端一样，可以在 sample-inputs 文件夹中为环境编辑一个示例存储类文件。将其复制到工作目录并进行必要的编辑，以反映所创建的后端。

```
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ cp sample-
input/storage-class-samples/storage-class-csi.yaml.templ ./storage-
class-basic.yaml
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ vi storage-class-
basic.yaml
```

5. 必须对此文件进行的唯一编辑是，为新创建的后端存储驱动程序的名称定义 `backendType` 值。另请注意

name-field 值，稍后必须引用该值。

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: basic-csi
provisioner: csi.trident.netapp.io
parameters:
  backendType: "ontap-san"
```

> 此文件中定义了一个名为 `FSType` 的可选字段。在iSCSI后端、可以将此值设置为特定的Linux文件系统类型(XFS、ext4等)、也可以删除此值、以便OpenShift决定要使用的文件系统。

6. 运行 `kubectl` 命令以创建存储类。

```
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ kubectl create -f
storage-class-basic.yaml
storageclass.storage.k8s.io/basic-csi created
```

7. 创建存储类后，您必须创建第一个永久性卷请求（ PVC ）。此外，还可以在 sample-inputs 中使用一个示例 `pva-basic` 。 `yaml` file 来执行此操作。

```
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ cp sample-input/pvc-
samples/pvc-basic.yaml ./
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ vi pvc-basic.yaml
```

8. 必须对此文件进行的唯一编辑是，确保 `storageClassName` 字段与刚刚创建的字段匹配。可以根据要配置的工作负载的需要进一步自定义 PVC 定义。

```
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: basic
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
  storageClassName: basic-csi
```

9. 发出 `kubectl` 命令创建 PVC 。根据所创建的后备卷的大小，创建可能需要一些时间，因此您可以在该过程

完成后进行观察。

```
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ kubectl create -f
pvc-basic.yaml
persistentvolumeclaim/basic created

[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ kubectl get pvc
NAME     STATUS   VOLUME                                     CAPACITY
ACCESS MODES   STORAGECLASS   AGE
basic    Bound    pvc-7ceac1ba-0189-43c7-8f98-094719f7956c   1Gi
RWO            basic-csi      3s
```

"接下来：高级配置选项：采用NetApp的Anthos。"

NetApp Element iSCSI配置：采用NetApp的Anthos

要启用 Trident 与 NetApp Element 存储系统的集成，您必须创建一个后端，以便使用 iSCSI 协议与存储系统进行通信。

1. 下载的安装归档中提供了 `sample-input` folder 层次结构中的示例后端文件。对于提供 iSCSI 服务的 NetApp Element 系统，将 `backend-solidfire.json` 文件复制到您的工作目录中，然后编辑该文件。

```
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ cp sample-
input/backends-samples/solidfire/backend-solidfire.json ./
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ vi ./backend-
solidfire.json
```

   a. 编辑 `endpoint` 行上的用户，密码和 MVIP 值。

   b. 编辑 `SVIP` 值。

```
 {
    "version": 1,
    "storageDriverName": "solidfire-san",
    "Endpoint": "https://trident:password@172.21.224.150/json-
rpc/8.0",
    "SVIP": "10.61.180.200:3260",
    "TenantName": "trident",
    "Types": [{"Type": "Bronze", "Qos": {"minIOPS": 1000, "maxIOPS":
2000, "burstIOPS": 4000}},
              {"Type": "Silver", "Qos": {"minIOPS": 4000, "maxIOPS":
6000, "burstIOPS": 8000}},
              {"Type": "Gold", "Qos": {"minIOPS": 6000, "maxIOPS":
8000, "burstIOPS": 10000}}]
}
```

2. 安装好此后端文件后，运行以下命令创建第一个后端。

```
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ ./tridentctl -n
trident create backend -f backend-solidfire.json
+------------------------+----------------
+--------------------------------------+--------+---------+
|          NAME          | STORAGE DRIVER |                  UUID
| STATE  | VOLUMES |
+------------------------+----------------
+--------------------------------------+--------+---------+
| solidfire_10.61.180.200 | solidfire-san  | b90783ee-e0c9-49af-8d26-
3ea87ce2efdf | online |       0 |
+------------------------+----------------
+--------------------------------------+--------+---------+
```

3. 创建后端后，您接下来必须创建一个存储类。与后端一样，可以在 sample-inputs 文件夹中为环境编辑一个示例存储类文件。将其复制到工作目录并进行必要的编辑，以反映所创建的后端。

```
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ cp sample-
input/storage-class-samples/storage-class-csi.yaml.templ ./storage-
class-basic.yaml
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ vi storage-class-
basic.yaml
```

4. 必须对此文件进行的唯一编辑是，为新创建的后端存储驱动程序的名称定义 `backendType` 值。另请注意 name-field 值，稍后必须引用该值。

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: basic-csi
provisioner: csi.trident.netapp.io
parameters:
  backendType: "solidfire-san"
```

> 此文件中定义了一个名为 `FSType` 的可选字段。在 iSCSI 后端，可以将此值设置为特定的 Linux 文件系统类型（ XFS ， ext4 等），也可以将其删除以允许 OpenShift 决定要使用的文件系统。

5. 运行 `kubectl` 命令以创建存储类。

```
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ kubectl create -f
storage-class-basic.yaml
storageclass.storage.k8s.io/basic-csi created
```

6. 创建存储类后，您必须创建第一个永久性卷请求（ PVC ）。此外，还可以在 sample-inputs 中使用一个示例 `pva-basic 。 yaml` file 来执行此操作。

```
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ cp sample-input/pvc-
samples/pvc-basic.yaml ./
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ vi pvc-basic.yaml
```

7. 必须对此文件进行的唯一编辑是，确保 `storageClassName` 字段与刚刚创建的字段匹配。可以根据要配置的工作负载的需要进一步自定义 PVC 定义。

```
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: basic
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
  storageClassName: basic-csi
```

8. 发出 `kubectl` 命令创建 PVC 。根据所创建的后备卷的大小，创建可能需要一些时间，因此您可以在该过程完成后进行观察。

```
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ kubectl create -f
pvc-basic.yaml
persistentvolumeclaim/basic created

[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ kubectl get pvc
NAME    STATUS   VOLUME                                     CAPACITY
ACCESS MODES   STORAGECLASS   AGE
basic   Bound    pvc-3445b5cc-df24-453d-a1e6-b484e874349d   1Gi
RWO            basic-csi      5s
```

"接下来：高级配置选项：采用NetApp的Anthos。"

Copyright Information

Copyright © 2022 NetApp, Inc. All rights reserved. Printed in the U.S. No part of this document covered by copyright may be reproduced in any form or by any means-graphic, electronic, or mechanical, including photocopying, recording, taping, or storage in an electronic retrieval system-without prior written permission of the copyright owner.

Software derived from copyrighted NetApp material is subject to the following license and disclaimer:

THIS SOFTWARE IS PROVIDED BY NETAPP “AS IS” AND WITHOUT ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE, WHICH ARE HEREBY DISCLAIMED. IN NO EVENT SHALL NETAPP BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

NetApp reserves the right to change any products described herein at any time, and without notice. NetApp assumes no responsibility or liability arising from the use of products described herein, except as expressly agreed to in writing by NetApp. The use or purchase of this product does not convey a license under any patent rights, trademark rights, or any other intellectual property rights of NetApp.

The product described in this manual may be protected by one or more U.S. patents, foreign patents, or pending applications.

RESTRICTED RIGHTS LEGEND: Use, duplication, or disclosure by the government is subject to restrictions as set forth in subparagraph (c)(1)(ii) of the Rights in Technical Data and Computer Software clause at DFARS 252.277-7103 (October 1988) and FAR 52-227-19 (June 1987).

Trademark Information

NETAPP, the NETAPP logo, and the marks listed at http://www.netapp.com/TM are trademarks of NetApp, Inc. Other company and product names may be trademarks of their respective owners.